PARTICLE SWARM OPTIMIZATION ALGORITHM FOR FINDING THE BRIGHTEST SPOT IN A GIVEN SPACE

A thesis submitted in partial fulfillment of the requirements for the award of the degree of

B.Tech

in

Instrumentation and Control Engineering

By

Ajay G Iyer (110115004)



INSTRUMENTATION AND CONTROL ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY
TIRUCHIRAPPALLI-620015

MAY 2019

BONAFIDE CERTIFICATE

This is to certify that the project titled **PARTICLE SWARM OPTIMIZATION ALGORITHM FOR FINDING THE BRIGHTEST SPOT IN A GIVEN SPACE** is a bonafide record of the work done by

Ajay G Iyer (110115004)

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Instrumentation and Control Engineering** of the **NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALLI**, during the year 2018-2019.

Guide

Head of the Department

ABSTRACT

Particle Swarm Optimization (PSO) algorithm is widely used in Robotic search applications. This paper proposes a PSO algorithm to locate the brightest spot in a given space using a swarm of robots. Each robot of the swarm acts as a particle in PSO; each particle updates its position and velocity, personal best location and personal best cost measurement based on the light intensity at any position till it reaches its target. The robots communicate the measured light intensity and its position with each other. The position of robots is obtained using ArUco markers with the help of an overhead camera. An ultrasonic sensor is interfaced with the robots for dynamic and static obstacle avoidance. Simulation of the PSO algorithm is implemented using MATLAB software.

Keywords: Personal best position; Personal best measurement; aruco marker

TABLE OF CONTENTS

Title **Page No.**

ABSTRACT . . . . . **ii**

TABLE OF CONTENTS . . . . . **iii**

LIST OF FIGURES . . . . . **v**

1 Introduction **1**

2 Hardware **3**

2.1 Lego Mindstorm NXT 3

2.1.1 NXT Intelligent Brick 3

2.1.2 NXT Programming 4

2.1.3 NXT-Python 4

2.1.4 Motor control of NXT robot to reach a given destination 5

3 ArUco Markers **7**

3.1 ArUco Markers 7

3.2 Dictionaries 9

3.3 Marker Creation 10

3.4 Marker Detection 11

3.5 Detection Process in ArUco 12

3.6 Pose Estimation 15

4 Particle Swarm Optimization **17**

4.1 Basic Particle Swarm Optimization 17

4.1.1 Global Best PSO 18

4.1.2 Local Best PSO 19

4.1.3 gbest versus lbest PSO 20

4.1.4 Velocity Components 20

4.1.5 Geometric Illustration 21

4.2 Implementation of Particle Swarm Optimization in MATLAB 23

4.2.1 Algorithm 23

4.2.2 Simulation result for PSO algorithm to find a target in the search space . . . 24
4.3 PSO algorithm to find the brightest spot . . . 25

5 Path Planning and Obstacle Detection . . . 26
5.1 Map representations . . . 26
5.2 Path-Planning Algorithms . . . 27
5.3 Robot embodiment . . . 29
5.4 Implementation . . . 29
5.4.1 A* Algorithm . . . 29
5.4.2 Obstacle detection and result . . . 31

6 Results and Observations . . . 34

7 Conclusion . . . 38

List of Figures

2.1 Lego 3
2.2 Navigate 5

3.1 ArUco Marker 8
3.2 Marker Class 8
3.3 ArUco Detection 9
3.4 Detecting midpoint of ArUco marker 10

4.1 22
4.2 22
4.3 Flowchart of PSO Algorithm 23
4.4 Global best cost vs number of iterations 24
4.5 Navigation of three robots to reach a target 24

5.1 A* Algorithm 30
5.2 Testcase 32
5.3 Output 33

6.1 Initial Setup 34
6.2 Iteration 1 34
6.3 Obstacle detection 35
6.4 Iteration 2 35
6.5 Iteration 3 36
6.6 Final Result 36
6.7 Time taken by three robots to reach the brightest spot 37

Chapter 1

Introduction

Particle Swarm Optimization technique is widely used for robotic search applications. There are several other techniques for path planning and optimization of mobile robots. Some of the popularly used approaches are Ant Colony Optimization Technique, Bacterial Foraging Optimization, Bee Colony Optimization Technique, Firefly Algorithm Optimization Technique, Grey Wolf Optimization.

Our primary objective is to build a swarm of robots that can navigate in a search space and locate the target. We have used LEGO Mindstorm NXT robots to implement this algorithm. Sensors are interfaced with the robots that can identify the target. There are many situations in which humans find it difficult to find the target or it can be dangerous for humans to find out. For instance, the target could be a bomb or radiation leak which is very difficult and dangerous for humans to figure it out. In all these cases robots can be used effectively.

These robots are small and less in weight so it will be easier to transport. A swarm of robots are cheaper and can search the area quickly lessening human risks. The swarm of robots does not have any master, slave communication. Each of the robots in the swarm calculates its own position and cost value and also communicates with other robots in the swarm.

The robots need to move in a definite way and not randomly search in the room to reach the target. There must be an algorithm which drives the robots to move from its initial position to the target. The algorithm is designed to encounter two main constraints:

- The algorithm should be designed in such a way that it is simple yet should handle simultaneous processes together. The position of the robots should be measured continuously throughout its navigation by an overhead camera with the help of ArUco markers. The robots must be controlled in such a way that it reaches a given destination.

- The communication traffic should be reduced within the robots. To encounter this we have used a central system which communicates with other robots in the swarm, compute all the PSO algorithm calculations and assigns the destination for each robot. If communication happens within the swarm, there can be a huge delay as each robot should wait for the other robot's information.

This thesis describes the PSO algorithm to find the brightest spot in an arena. In section 2 we have described the hardware we have used and all the sensor's interaction. In section 3 we have reviewed the PSO algorithm, designed and simulated a PSO algorithm to find a target and the PSO algorithm to find the brightest spot have been described and implemented in real time. Section 4 provides the results and observations from the experimental analysis. The results show that the swarm is able to find the target. Section 5 gives the conclusions for this thesis.

Chapter 2

Hardware

2.1 Lego Mindstorm NXT

LEGO Mindstorms NXT is a programmable robotics kit released by Lego in late July 2006. It replaced the first-generation Lego Mindstorms kit, which was called the Robotics Invention System. It comes with the NXT-G programming software, or optionally LabVIEW for Lego Mindstorms. A variety of unofficial languages exist, such as NXC, NBC, leJOS NXJ, and RobotC.

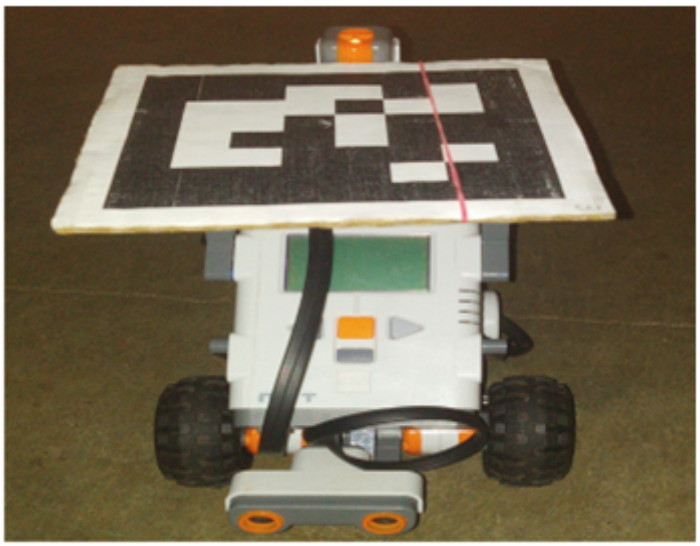


Figure 2.1: Lego

2.1.1 NXT Intelligent Brick

The main component in the kit is a brick-shaped computer called the NXT Intelligent Brick. It can take input from up to four sensors and control up to three motors, via a modified version of RJ12 cables, very much similar to but incompatible with RJ11 phone cords. The plastic pin to hold the cable in the socket is moved slightly to the

right. The brick has a 10060 pixel monochrome LCD and four buttons that can be used to navigate a user interface using hierarchical menus. It has a 32-bit ARM7TDMI-core AtmelAT91SAM7S256 microcontroller with 256 KB of FLASH memory and 64 KB of RAM, plus an 8-bit Atmel AVR ATmega48 microcontroller, and bluetooth support. It also has a speaker and can play sound files at sampling rates up to 8 kHz. Power is supplied by 6 AA (1.5 V each) batteries in the consumer version of the kit and by a Li-Ion rechargeable battery and charger in the educational version.

2.1.2 NXT Programming

Very simple programs can be created using the menu on the NXT Intelligent Brick. More complicated programs and sound files can be downloaded using a USB port or wirelessly using Bluetooth. Files can also be copied between two NXT bricks wirelessly, and some mobile phones can be used as a remote control. Up to three NXT bricks can communicate simultaneously via Bluetooth when users created programs are run.

2.1.3 NXT-Python

NXT-Python is a package for controlling a LEGO NXT robot using the Python programming language. It can communicate using either USB or Bluetooth.
The Lego Mindstorms NXT 1.0 base kit includes:

- Identical servo motors that have built-in reduction gear assemblies with internal optical rotary encoders that sense their rotations within one degree of accuracy.
- The touch sensor detects whether it is currently pressed, has been bumped, or released. The orange Enter button and the gray right and left NXT buttons can be programmed to serve as touch sensors. In the NXT-G programming software, a value of 0 is given out when it is not pressed, and a value of 1 is given out if it is pressed down.
- The light sensor detects the light level in one direction, and also includes a LED for illuminating an object. The light sensor can sense reflected light values (using the built-in red LED), or ambient light. In the NXT-G programming software the sensor senses light on a scale of 0 to 100, 100 being very bright and 0 being dark. If calibrated, the sensor can also be used as a distance sensor.
- The sound sensor measures volume level on a scale of 0 to 100, 100 being very loud, 0 being completely silent.
- The ultrasonic sensor can measure the distance from the sensor to something that it is facing, and detect movement. It can show the distance in both centimeters

and inches. The maximum distance it can measure is 233 cm with a precision of 3 centimeters. The ultrasonic sensor works by sending out ultrasonic sound waves that bounce off an object ahead of it and then back. It senses the time it took for that to happen. In the Lego Mindstorms 2.0 base kit, it includes: 2 Touch sensors, one Color sensor (detects several different colors), and an Ultrasonic sensor.

These parts are not included in the Lego Mindstorms NXT base kit and may be bought separately:

- Third-party companies also manufacture sensors such as the compass, gyroscope, infrared tracker, RFID reader and accelerometer sensors sold by Lego.
- The temperature sensor can measure temperature in Celsius or Fahrenheit.

The brick of the NXT robots are connected to the central system via bluetooth. The robots are built in such a way that it can move in all directions without any jerks. It accommodates aruco marker on the top, ultrasonic sensor in the front to detect obstacles and a light sensor to measure the intensity of light. The motors are connected to PORT A and C, ultrasonic sensor and light sensor are connected to PORT 1 and 3 of the brick respectively. The ArUco marker is placed in such a way that the position of each robot is detected continuously by an overhead camera. This camera is connected to the system via USB. The arena is set in such a way that there is one brightest spot, an overhead camera and three robots placed at random positions.

2.1.4 Motor control of NXT robot to reach a given destination

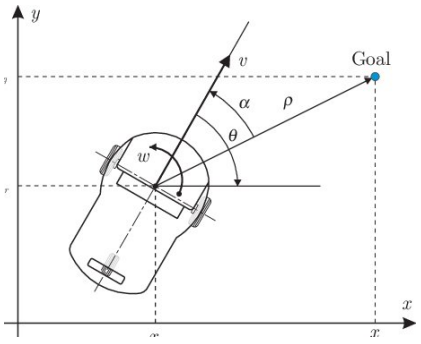


Figure 2.2: Navigate

The robot when given a coordinate follows certain rules to reach the destination. First, the robot must orient itself towards the destination. From Fig.2.2 we can observe that θ is the robot's orientation with the axis, α is the angle between the robot and the destination. For the robot to reach the destination, the robot must turn at an angle of θ - α to orient itself towards the destination point. After the robot orients itself towards

the target, it moves the required distance to reach the given coordinate. The change in orientation, movement of the robot must be monitored continuously by the overhead camera.

Chapter 3

ArUco Markers

Pose estimation is of great importance in many computer vision applications: robot navigation, augmented reality, and many more. This process is based on finding correspondences between points in the real environment and their 2d image projection. This is usually a difficult step, and thus it is common the use of synthetic or fiducial markers to make it easier.

One of the most popular approach is the use of binary square fiducial markers. The main benefit of these markers is that a single marker provides enough correspondences (its four corners) to obtain the camera pose. Also, the inner binary codification makes them especially robust, allowing the possibility of applying error detection and correction techniques.

A fiducial marker or fiducial is an object placed in the field of view of an imaging system which appears in the image produced, for use as a point of reference or a measure. Fiducials of known pattern and size can serve as real world anchors of location, orientation and scale. They can establish the identity of the scene or objects within the scene.

3.1 ArUco Markers

An ArUco marker is a synthetic square marker composed by a wide black border and an inner binary matrix which determines its identifier (id). The black border facilitates its fast detection in the image and the binary codification allows its identification and the application of error detection and correction techniques. The marker size determines the size of the internal matrix.

By default, the MarkerDetectorwill look for squares and then will analyze the binary code inside. For the code extracted, it will compare against all the markers in all the

available dictionaries.

Markers are comprised by an external black border and an inner region that encodes a binary pattern. The binary pattern is unique and identifies each marker. Depending on the dictionary, there are markers with more or fewer bits.

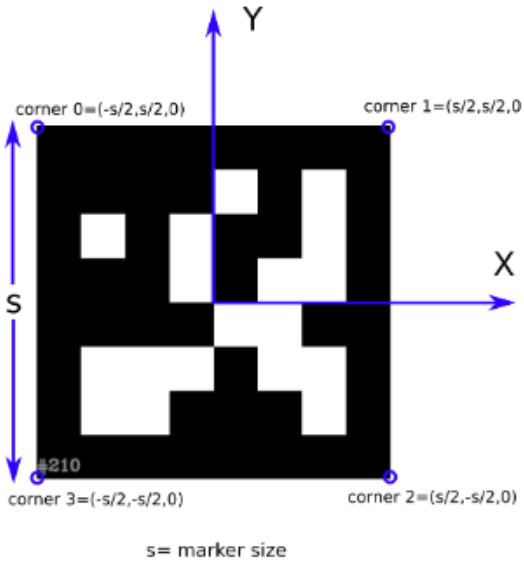


Figure 3.1: ArUco Marker

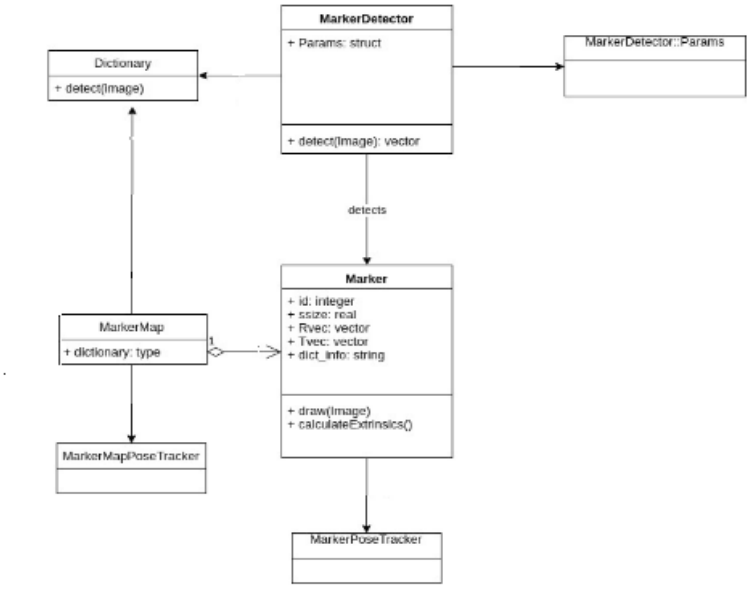


Figure 3.2: Marker Class

- Marker - represents a marker.
- Marker Detector - does the detection work.

  Params - parameters controlling the different aspect of detection.
- Dictionary - defines the set of binary patterns.
- Markermap - A set of markers along with its 3D.
- MarkerPoseTracker - tracks the pose of a marker in a video sequence.
- MarkerMapPoseTracker - tracks the pose of the camera in a video sequence with respect to a marker map.

3.2 Dictionaries

A dictionary of markers is a set of markers that are considered in an specific application. It is simply the list of binary codifications of each of its markers.

The main properties of a dictionary are the dictionary size and the marker size.

- The dictionary size is the number of markers that composed the dictionary.
- The marker size is the size of those markers (the number of bits).

The ArUco module includes some predefined dictionaries covering a range of different dictionary sizes and marker sizes.
A marker id is simply the marker index inside the dictionary it belongs to. For instance, the first 5 markers inside a dictionary have the ids: 0, 1, 2, 3 and 4.

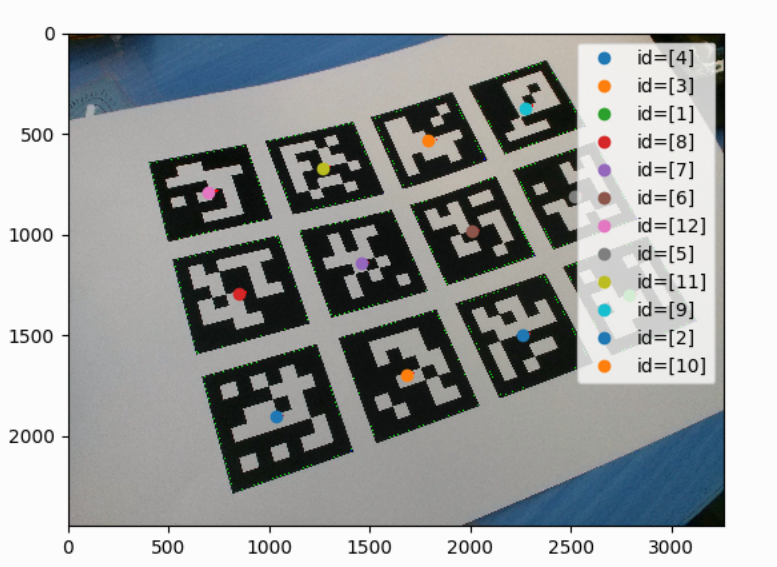


Figure 3.3: ArUco Detection

It must be noted that a marker can be found rotated in the environment, however, the detection process needs to be able to determine its original rotation, so that each corner is identified unequivocally.

		c1	c2	c3	c4	m1	m2	m3	m4	o
marker										
1	x	3114.0	2701.0	2467.0	2876.0	2907.5	2584.0	2671.5	2995.0	2789.50
	y	1429.0	1597.0	1168.0	1019.0	1513.0	1382.5	1093.5	1224.0	1303.25
2	x	2593.0	2152.0	1939.0	2363.0	2372.5	2045.5	2151.0	2478.0	2261.75
	y	1635.0	1804.0	1352.0	1209.0	1719.5	1578.0	1280.5	1422.0	1500.00
3	x	2037.0	1533.0	1350.0	1826.0	1785.0	1441.5	1588.0	1931.5	1686.50
	y	1848.0	2032.0	1518.0	1381.0	1940.0	1775.0	1449.5	1614.5	1694.75
4	x	1409.0	822.0	670.0	1231.0	1115.5	746.0	950.5	1320.0	1033.00
	y	2076.0	2281.0	1712.0	1553.0	2178.5	1996.5	1632.5	1814.5	1905.50
5	x	2820.0	2415.0	2217.0	2614.0	2617.5	2316.0	2415.5	2717.0	2516.50
	y	924.0	1071.0	686.0	550.0	997.5	878.5	618.0	737.0	807.75
6	x	2316.0	1883.0	1705.0	2121.0	2099.5	1794.0	1913.0	2218.5	2006.25
	y	1105.0	1248.0	860.0	720.0	1176.5	1054.0	790.0	912.5	983.25
7	x	1779.0	1311.0	1154.0	1603.0	1545.0	1232.5	1378.5	1691.0	1461.75
	y	1279.0	1409.0	989.0	886.0	1344.0	1199.0	937.5	1082.5	1140.75
8	x	1193.0	640.0	525.0	1039.0	916.5	582.5	782.0	1116.0	849.25
	y	1439.0	1592.0	1133.0	1013.0	1515.5	1362.5	1073.0	1226.0	1294.25
9	x	2561.0	2173.0	1998.0	2374.0	2367.0	2085.5	2186.0	2467.5	2276.50
	y	464.0	598.0	272.0	146.0	531.0	435.0	209.0	305.0	370.00
10	x	2068.0	1667.0	1519.0	1902.0	1867.5	1593.0	1710.5	1985.0	1789.00
	y	628.0	762.0	428.0	309.0	695.0	595.0	368.5	468.5	531.75
11	x	1563.0	1119.0	987.0	1411.0	1341.0	1053.0	1199.0	1487.0	1270.00
	y	797.0	896.0	543.0	449.0	846.5	719.5	496.0	623.0	671.25
12	x	1008.0	501.0	407.0	881.0	754.5	454.0	644.0	944.5	699.25
	y	920.0	1033.0	651.0	563.0	976.5	842.0	607.0	741.5	791.75

Figure 3.4: Detecting midpoint of ArUco marker

3.3 Marker Creation

Before their detection, markers need to be printed in order to be placed in the environment. Marker images can be generated using the drawMarker() function.

The parameters of drawMarker are:

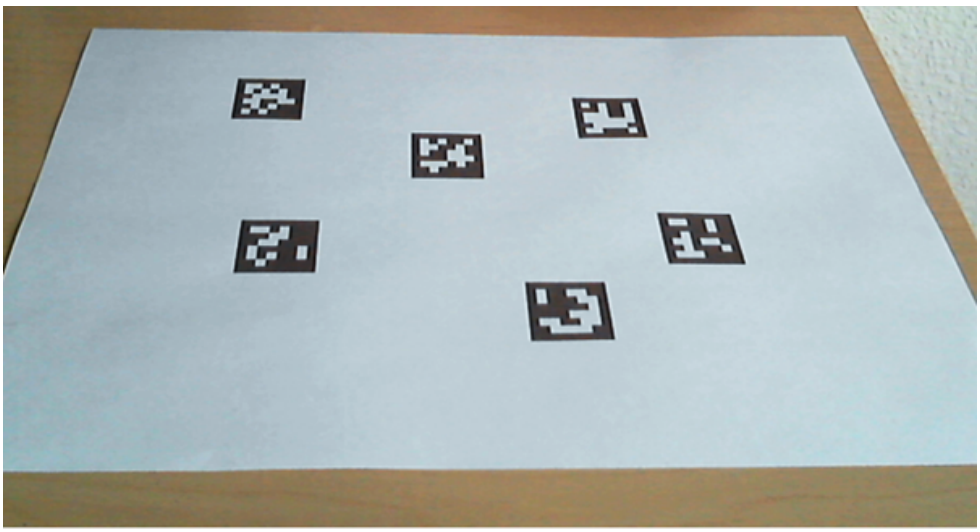
- The first parameter is the Dictionary object previously created.

- The second parameter is the marker id.Each dictionary is composed by a different number of markers. Any specific id out of the valid range will produce an exception.
- The third parameter is the size of the output marker image. This parameter should be large enough to store the number of bits for the specific dictionary. So, for instance, you cannot generate an image of 5x5 pixels for a marker size of 6x6 bits (and that is without considering the marker border). Furthermore, to avoid deformations, this parameter should be proportional to the number of bits + border size, or at least much higher than the marker size so that deformations are insignificant.
- The forth parameter is the output image.
- Finally, the last parameter is an optional parameter to specify the width of the marker black border. The size is specified proportional to the number of bits. For instance a value of 2 means that the border will have a width equivalent to the size of two internal bits. The default value is 1.

3.4 Marker Detection

Given an image where some ArUco markers are visible, the detection process has to return a list of detected markers.
Each detected marker includes:

- The position of its four corners in the image (in their original order).
- The id of the marker.



Original image with markers

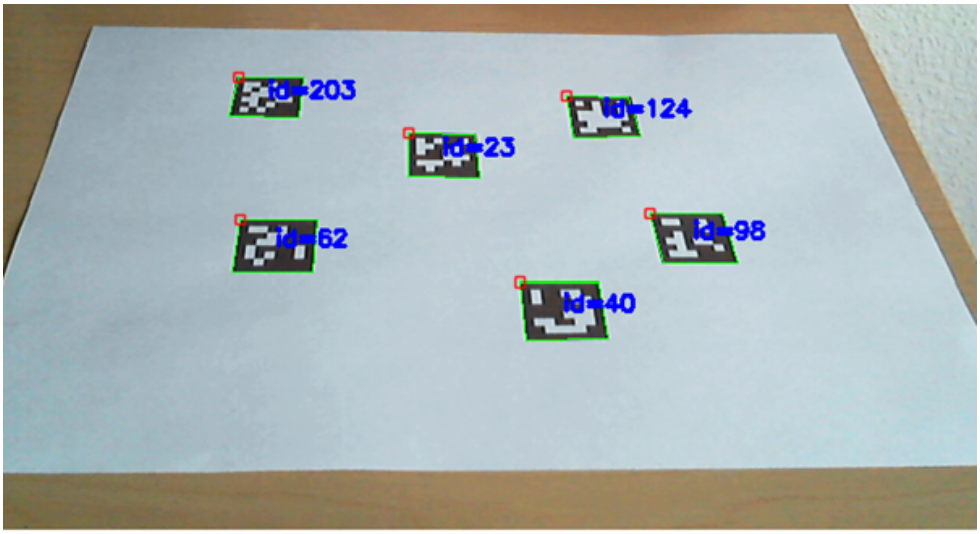


Image with detected markers

In the aruco module, the detection is performed in the detectMarkers() function. The parameters of detectMarkers are:

- The first parameter is the image where the markers are going to be detected.
- The second parameter is the dictionary object.
- The detected markers are stored in the markerCorners and markerIds structures:
  - markerCorners is the list of corners of the detected markers. For each marker, its four corners are returned in their original order (which is clockwise starting with top left). So, the first corner is the top left corner, followed by the top right, bottom right and bottom left.
  - markerIds is the list of ids of each of the detected markers in markerCorners. Note that the returned markerCorners and markerIds vectors have the same sizes.

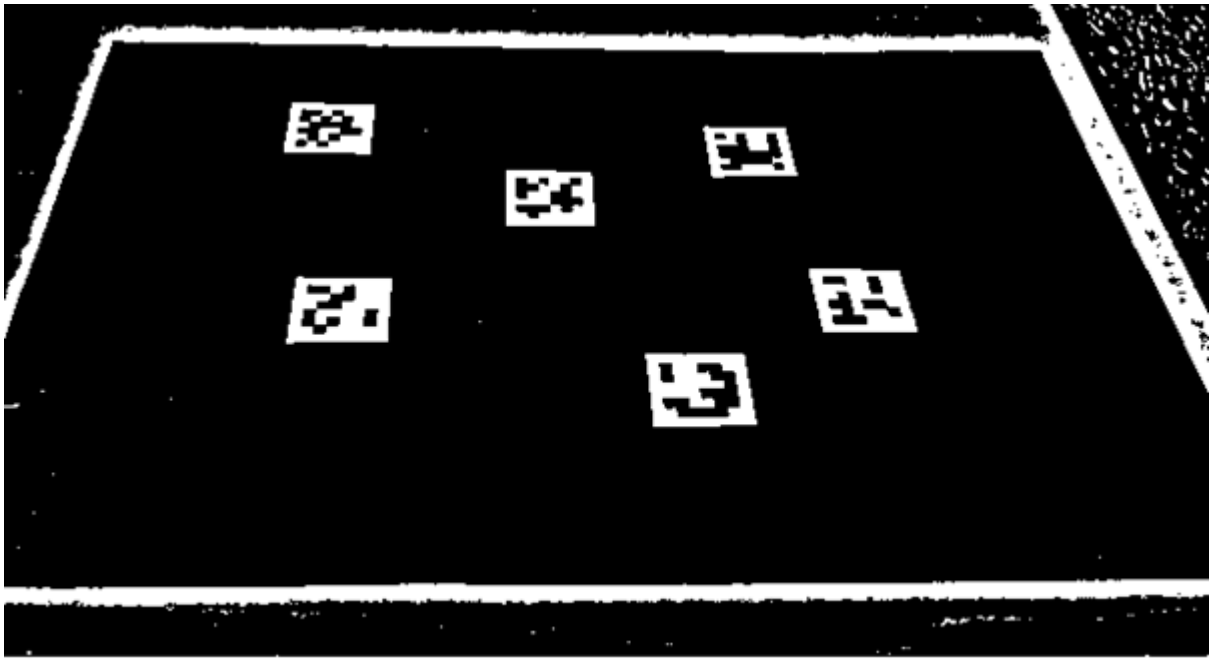
3.5 Detection Process in ArUco

The marker detection process of ArUco is as follows:

- Thresholding

  One of the first steps of the marker detection process is an adaptive thresholding of the input image.

For instance, the thresholded image for the sample image shown in Fig.3.5



Thresholded image

- Contour filtering

  After thresholding, contours are detected. However, not all contours are considered as marker candidates. They are filtered out in different steps so that contours that are very unlikely to be markers are discarded.

  It must be noted that in most cases it is a question of balance between detection capacity and performance. All the considered contours will be processed in the following stages, which usually have a higher computational cost. So, it is preferred to discard wrong candidates in this stage than in the later stages.

- Bits Extraction

  After candidate detection, the bits of each candidate are analyzed in order to determine if they are markers or not.

  Before analyzing the binary code itself, the bits need to be extracted. To do so, the perspective distortion is removed and the resulting image is thresholded using Otsu threshold to separate black and white pixels.

  Otsu's algorithm assumes a bimodal distribution and finds the threshold that maximizes the extra-class variance while keeping a low intra-class variance.

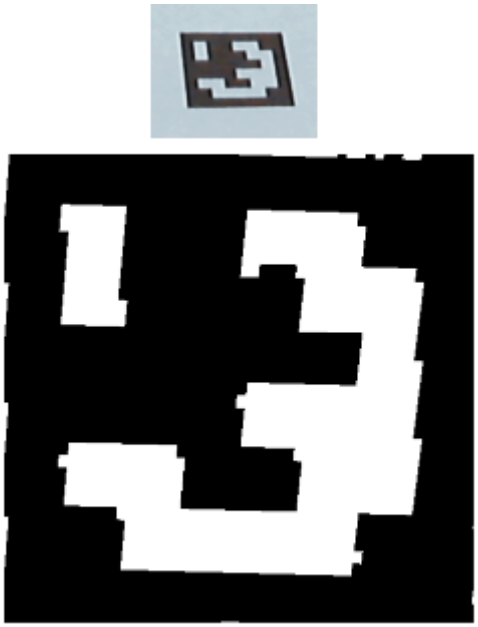
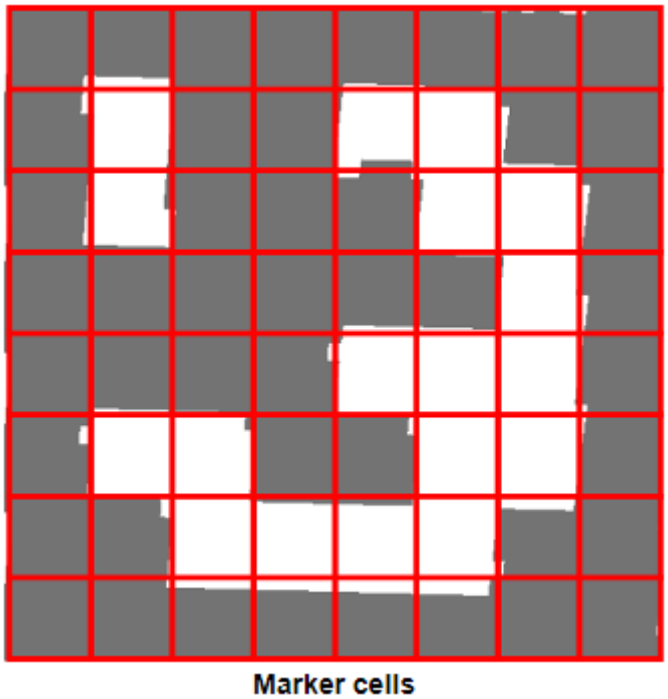


Fig.3.5 obtained after removing the perspective distortion of a marker.

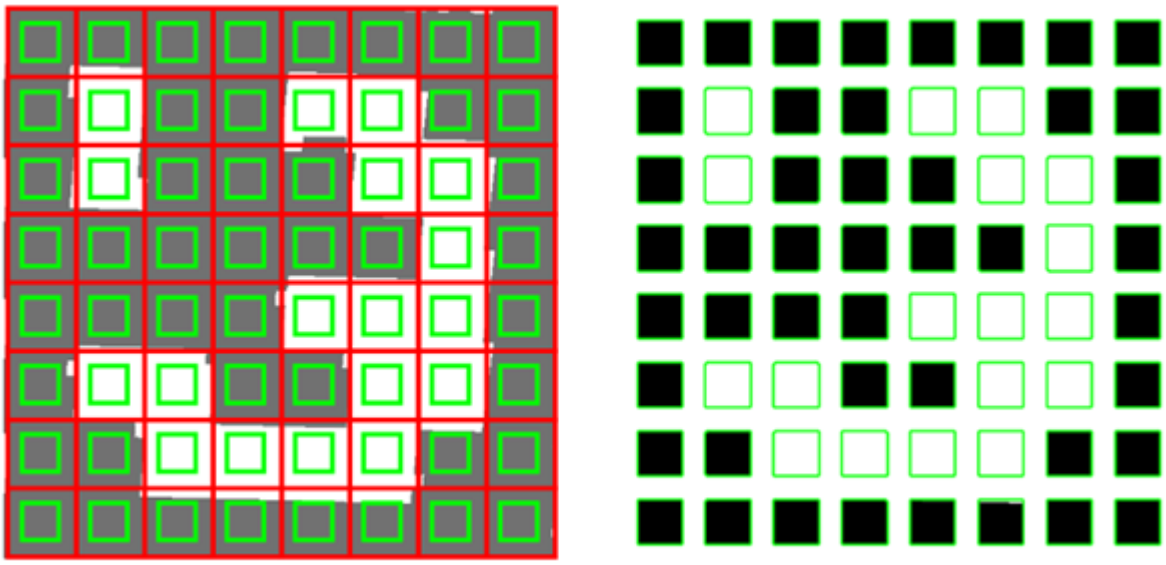
Then, the image is divided in a grid with the same cells than the number of bits in the marker. On each cell, the number of black and white pixels is counted to decide the bit assigned to the cell.



When extracting the bits of each cell, the numbers of black and white pixels are counted. In general, it is not recommended to consider all the cell pixels. Instead it is better to

ignore some pixels in the margins of the cells.

The reason of this is that, after removing the perspective distortion, the cells' colors are, in general, not perfectly separated and white cells can invade some pixels of black cells (and vice-versa). Thus, it is better to ignore some pixels just to avoid counting erroneous pixels.



For instance, in the above image only the pixels inside the green squares are considered. It can be seen in the right image that the resulting pixels contain a lower amount of noise from neighbour cells.

- Marker identification

  After the bits have been extracted, the next step is checking if the extracted code belongs to the marker dictionary and, if necessary, error correction can be performed.

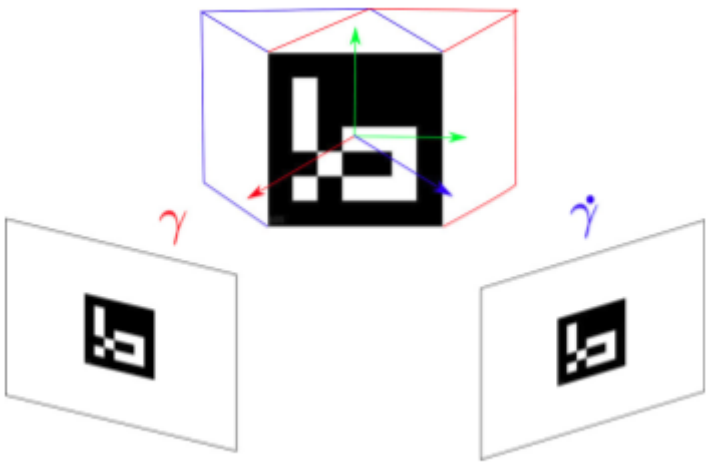
- Corner Refinement

  After markers have been detected and identified, the last step is performing sub-pixel refinement in the corner positions.

3.6 Pose Estimation

The next thing you probably want to do after detecting the markers is to obtain the camera pose from them. The camera pose respect to a marker is the 3d transformation from the marker coordinate system to the camera coordinate system. It is specified by a rotation and a translation vector.

The following image shows the 3D reference system of each marker. The detection of the four corners of a marker, allows applying planar pose estimators. They estimate the relative pose of the camera with respect to the center of the marker.



The ambiguity problem. The same marker projection could come from two poses, the two cubes shown in red and blue.

It is extremely important to remark that the estimation of the pose using only 4 coplanar points is subject to ambiguity. As shown in the above figure, a marker could project at the same pixels on two different camera locations. In general, the ambiguity can be solved, if the camera is near to the marker. However, as the marker becomes small, the error in the corner estimation grows and ambiguity comes as a problem.

Chapter 4

Particle Swarm Optimization

The particle swarm optimization (PSO) algorithm is a population-based search algorithm based on the simulation of the social behavior of birds within a flock. The initial intent of the particle swarm concept was to graphically simulate the graceful and unpredictable choreography of a bird flock, with the aim of discovering patterns that govern the ability of birds to fly synchronously, and to suddenly change direction with a regrouping in an optimal formation. From this initial objective, the concept evolved into a simple and efficient optimization algorithm.

In PSO, individuals, referred to as particles, are "flown" through hyperdimensional search space. Changes to the position of particles within the search space are based on the social-psychological tendency of individuals to emulate the success of other individuals. The changes to a particle within the swarm are therefore influenced by the experience, or knowledge, of its neighbors. The search behavior of a particle is thus affected by that of other particles within the swarm (PSO is therefore a kind of symbiotic cooperative algorithm). The consequence of modeling this social behavior is that the search process is such that particles stochastically return toward previously successful regions in the search space.

4.1 Basic Particle Swarm Optimization

Individuals in a particle swarm follow a very simple behavior: to emulate the success of neighboring individuals and their own successes. The collective behavior that emerges from this simple behavior is that of discovering optimal regions of a high dimensional search space.

A PSO algorithm maintains a swarm of particles, where each particle represents a potential solution. In analogy with evolutionary computation paradigms, a swarm is similar to a population, while a particle is similar to an individual. In simple terms, the

particles are “flown” through a multidimensional search space, where the position of each particle is adjusted according to its own experience and that of its neighbors. Let xi(t) denote the position of particle i in the search space at time step t; unless otherwise stated, t denotes discrete time steps. The position of the particle is changed by adding a velocity, vi(t), to the current position, i.e.

$$x_i(t+1) = x_i(t) + v_i(t)$$

It is the velocity vector that drives the optimization process, and reflects both the experiential knowledge of the particle and socially exchanged information from the particle's neighborhood. The experiential knowledge of a particle is generally referred to as the cognitive component, which is proportional to the distance of the particle from its own best position (referred to as the particle's personal best position) found since the first time step. The socially exchanged information is referred to as the social component of the velocity equation.

4.1.1 Global Best PSO

For the global best PSO, or gbest PSO, the neighborhood for each particle is the entire swarm. The social network employed by the gbest PSO reflects the star topology . For the star neighborhood topology, the social component of the particle velocity update reflects information obtained from all the particles in the swarm. In this case, the social information is the best position found by the swarm, referred to as g(t).
For gbest PSO, the velocity of particle i is calculated as

$$v_{ij}(t+1) = (v_{ij}(t) + c1 * r_{1j}(t) * (y_{ij}(k) - x_{ij}) + (c2 * r_{2j}(t) * (g_{ij}(k) - x_{ij}(k))$$

where vij(t) is the is the velocity of particle i in dimension j = 1,...,nx at time step t, xij(t) is the position of particle i in dimension j at time step t, c1 and c2 are positive acceleration constants used to scale the contribution of the cognitive and social components respectively. r1j (t), r2j(t) are random values in the range [0, 1], sampled from a uniform distribution. These random values introduce a stochastic element to the algorithm. The personal best position, yi, associated with particle i is the best position the particle has visited since the first time step. Considering minimization problems, the personal best position at the next time step, t + 1, is calculated as

$$\mathbf{y}_i(t+1) = \begin{cases} \mathbf{y}_i(t) & \text{if } f(\mathbf{x}_i(t+1)) \geq f(\mathbf{y}_i(t)) \\ \mathbf{x}_i(t+1) & \text{if } f(\mathbf{x}_i(t+1)) < f(\mathbf{y}_i(t)) \end{cases}$$

The global best position, g(t), at time step t, is defined as g(t)

$$g_0(t), ..., g_{ns}(t) | f(g(t)) = minf(g_0(t)), ..., f(g_{ns}(t))$$

where ns is the total number of particles in the swarm. It is important to note that the definition in equation states that g is the best position discovered by any of the particles so far – it is usually calculated as the best personal best position. The global best position can also be selected from the particles of the current swarm, in which case

$$g(t) = min(f((x_0(t)), ..., f(x_n(t))$$

The *gbest* PSO is summarized in Algorithm 16.1.

Algorithm 16.1 *gbest* PSO

Create and initialize an n_x-dimensional swarm;
repeat
  for *each particle* $i = 1, \ldots, n_s$ **do**
    //set the personal best position
    if $f(\mathbf{x}_i) < f(\mathbf{y}_i)$ **then**
      $\mathbf{y}_i = \mathbf{x}_i$;
    end
    //set the global best position **if** $f(\mathbf{y}_i) < f(\hat{\mathbf{y}})$ **then**
      $\hat{\mathbf{y}} = \mathbf{y}_i$;
    end
  end
  for *each particle* $i = 1, \ldots, n_s$ **do**
    update the velocity using equation (16.2);
    update the position using equation (16.1);
  end
until *stopping condition is true*;

4.1.2 Local Best PSO

The local best PSO, or lbest PSO, uses a ring social network topology where smaller neighborhoods are defined for each particle. The social component reflects information exchanged within the neighborhood of the particle, reflecting local knowledge of the environment. With reference to the velocity equation, the social contribution to particle velocity is proportional to the distance between a particle and the best position found by the neighborhood of particles. The velocity is calculated as

$$v_{ij}(t+1) = (v_{ij}(t) + c1 * r_{1j}(t) * (y_{ij}(k) - x_{ij}) + (c2 * r_{2j}(t) * (g_{ij}(k) - x_{ij}(k))$$

where gij is the best position, found by the neighborhood of particle i in dimension j. The local best particle position, gi, i.e. the best position found in the neighborhood Ni. The local best position will also be referred to as the neighborhood best position. It is important to note that for the basic PSO, particles within a neighborhood have no relationship to each other. Selection of neighborhoods is done based on particle indices. However, strategies have been developed where neighborhoods are formed based on spatial similarity. Fig.4.1.2 summarizes lbest PSO.

Algorithm 16.2 *lbest* PSO

Create and initialize an n_x-dimensional swarm;
repeat
  for *each particle* $i = 1, \ldots, n_s$ **do**
    //set the personal best position
    if $f(\mathbf{x}_i) < f(\mathbf{y}_i)$ **then**
      $\mathbf{y}_i = \mathbf{x}_i$;
    end
    //set the neighborhood best position
    if $f(\mathbf{y}_i) < f(\hat{\mathbf{y}}_i)$ **then**
      $\hat{\mathbf{y}} = \mathbf{y}_i$;
    end
  end
  for *each particle* $i = 1, \ldots, n_s$ **do**
    update the velocity using equation (16.6);
    update the position using equation (16.1);
  end
until *stopping condition is true*;

4.1.3 gbest versus lbest PSO

The two versions of PSO discussed above are similar in the sense that the social component of the velocity updates causes both to move towards the global best particle. This is possible for the lbest PSO due to the overlapping neighborhoods.
There are two main differences between the two approaches with respect to their convergence characteristics

- Due to the larger particle interconnectivity of the gbest PSO, it converges faster than the lbest PSO. However, this faster convergence comes at the cost of less diversity than the lbest PSO.
- As a consequence of its larger diversity (which results in larger parts of the search space being covered), the lbest PSO is less susceptible to being trapped in local minima. In general (depending on the problem), neighborhood structures such as the ring topology used in lbest PSO improves performance.

4.1.4 Velocity Components

The velocity calculation as given in equations consists of three terms:

- The previous velocity, v(t), which serves as a memory of the previous flight direction, i.e. movement in the immediate past. This memory term can be seen as a momentum, which prevents the particle from drastically changing direction, and to bias towards the current direction. This component is also referred to as the inertia component.

- The cognitive component, quantifies the performance of particle i relative to past performances. In a sense, the cognitive component resembles individual memory of the position that was best for the particle. The effect of this term is that particles are drawn back to their own best positions, resembling the tendency of individuals to return to situations or places that satisfied them most in the past. Kennedy and Eberhart also referred to the cognitive component as the "nostalgia" of the particle.

- The social component quantifies the performance of particle i relative to a group of particles, or neighbors. Conceptually, the social component resembles a group norm or standard that individuals seek to attain. The effect of the social component is that each particle is also drawn towards the best position found by the particle's neighborhood.

The contribution of the cognitive and social components are weighed by a stochastic amount, c1r1 or c2r2, respectively.

4.1.5 Geometric Illustration

The effect of the velocity equation can easily be illustrated in a two-dimensional vector space. For the sake of the illustration, consider a single particle in a two-dimensional search space.

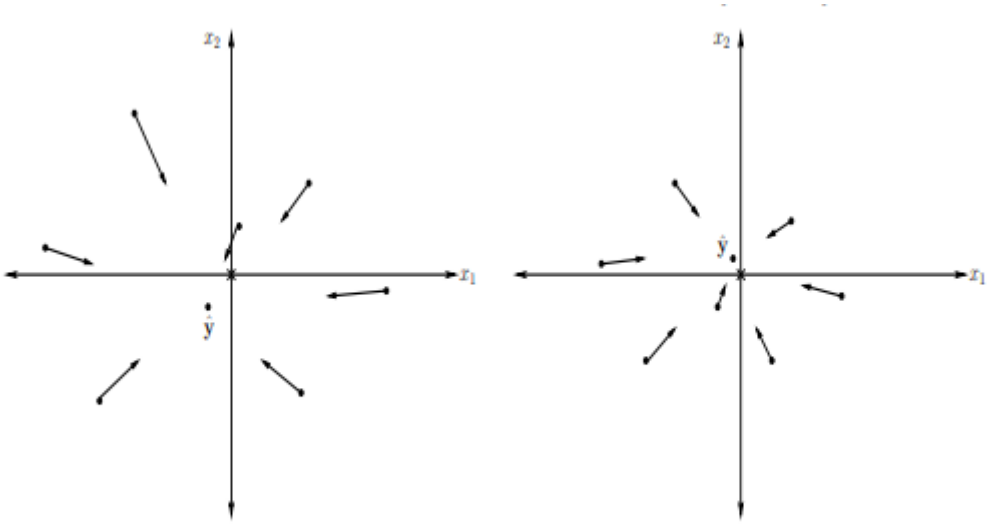
It is of course possible for a particle to overshoot the global best position, mainly due to the momentum term. This results in two scenarios:

1. The new position, as a result of overshooting the current global best, may be a better position than the current global best. In this case the new particle position will become the new global best position, and all particles will be drawn towards it.

2. The new position is still worse than the current global best particle. In subsequent time steps the cognitive and social components will cause the particle to change direction back towards the global best.

The cumulative effect of all the position updates of a particle is that each particle converges to a point on the line that connects the global best position and the personal best position of the particle.

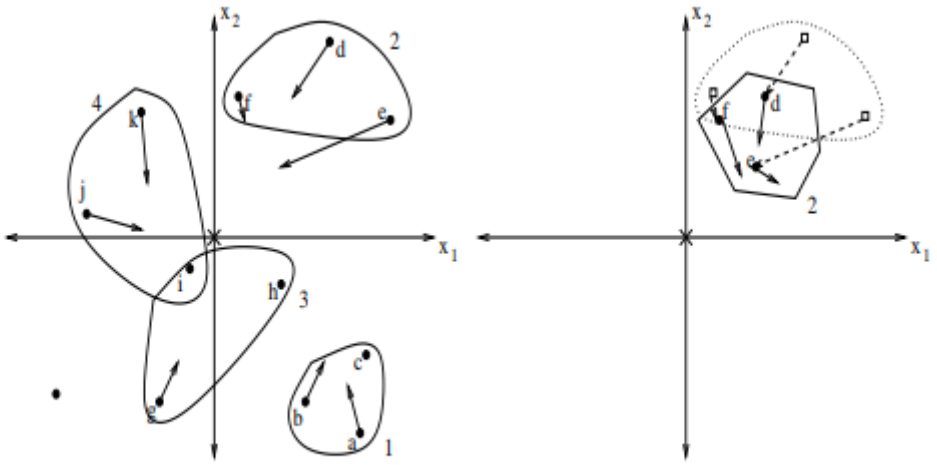
Returning to more than one particle, Figure 4.1 visualizes the position updates with reference to the task of minimizing a two-dimensional function with variables x1 and x2 using gbest PSO. The optimum is at the origin, indicated by the symbol . Figure 4.1a) illustrates the initial positions of eight particles, with the global best position as indicated. Since the contribution of the cognitive component is zero for each particle



(a) At time $t = 0$ (b) At time $t = 1$

Figure 4.1:

at time step t = 0, only the social component has an influence on the position adjustments. Note that the global best position does not change (it is assumed that vi(0) = 0, for all particles). Figure 4.1(b) shows the new positions of all the particles after the first iteration. A new global best position has been found. Figure 4.1(b) now indicates the influence of all the velocity components, with particles moving towards the new global best position.



(a) Local Best Illustrated – Initial Swarm (b) Local Best – Second Swarm

Figure 4.2:

Finally, the lbest PSO, as illustrated in Figure 16.3, shows how particles are influenced by their immediate neighbors. To keep the graph readable, only some of the movements are illustrated, and only the aggregate velocity direction is indicated. In neighborhood 1, both particles a and b move towards particle c, which is the best solution within that neighborhood. Considering neighborhood 2, particle d moves towards f, so does e. For the next iteration, e will be the best solution for neighborhood 2. Now d and f move towards e as illustrated in Figure 4.2 (only part of the solution space is illustrated). The blocks represent the previous positions. Note that e remains the best solution for neighborhood 2. Also note the general movement towards the minimum.

4.2 Implementation of Particle Swarm Optimization in MATLAB

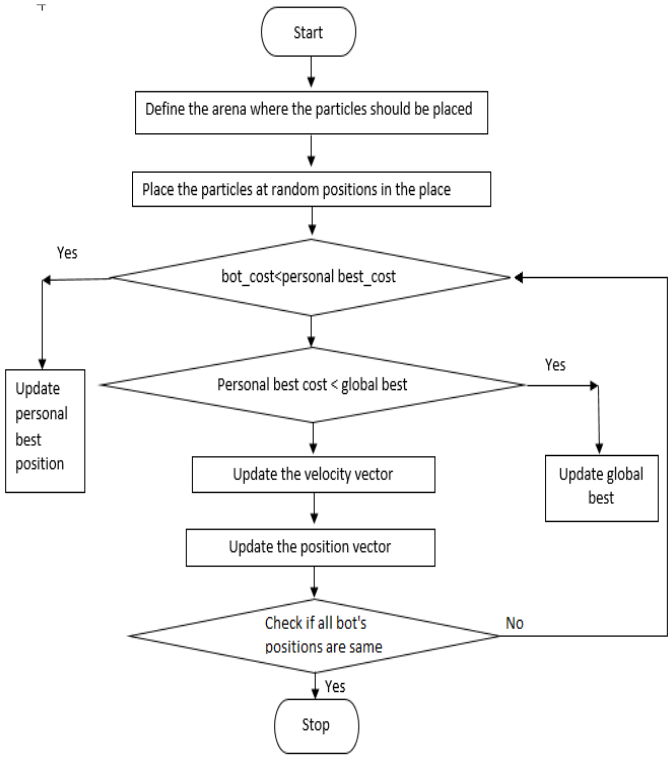


Figure 4.3: Flowchart of PSO Algorithm

4.2.1 Algorithm

Step 1 : Define the arena where the particles are placed. Step 2 : Place the particles at random positions in the search space. Step 3 : Determine the personal best position of each particle and update the personal best value. Step 4 : Determine the global best position among all the particles in the swarm and update the global best value.
Step 5 : Update the velocities of each particle in the swarm using Eq.(1).

Step 6 : Update the positions of each particle in the swarm using Eq.(2).
Step 7 : Repeat Steps 3 to 6 till the position of all three robots are same.

4.2.2 Simulation result for PSO algorithm to find a target in the search space

Here the target has been set as the coordinates (4,6). We have taken 3 particles in the swarm in a 2D search space. The constants c1, c2 and w are taken to be 1.5, 1.5, 0.3 respectively. The global best cost plot vs number of iterations and the navigation plot of the three particles for 500 iterations are shown below.

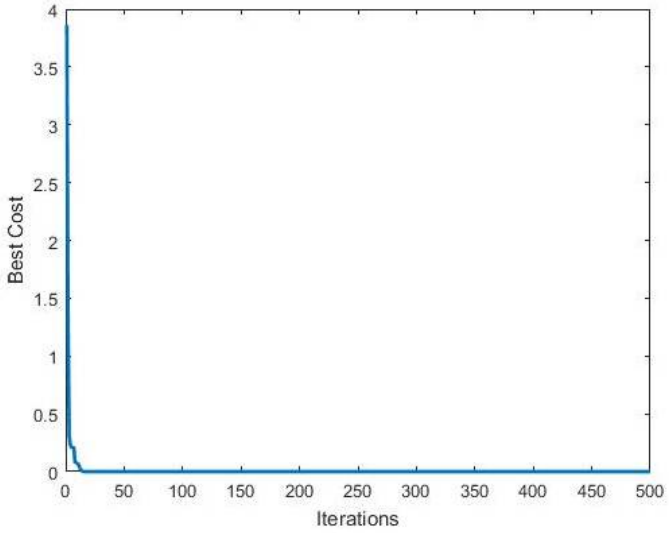


Figure 4.4: Global best cost vs number of iterations

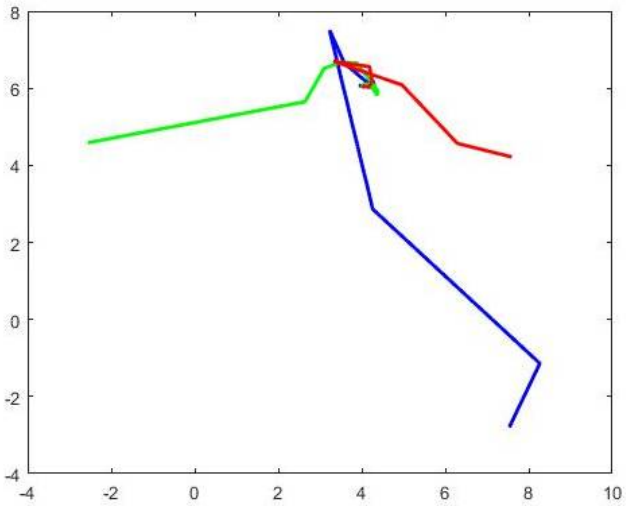


Figure 4.5: Navigation of three robots to reach a target

4.3 PSO algorithm to find the brightest spot

When this algorithm is implemented in real time the personal best and the global best values are obtained by the intensity values sensed by the light sensor of each robot. The personal best value is the highest intensity value sensed by each robot till its current state. Global best value is the highest intensity value of all the robots in the swarm. The algorithm is continued till all the particles in the swarm sense the same highest intensity value.

Chapter 5

Path Planning and Obstacle Detection

Path-planning is an important primitive for autonomous mobile robots that lets robots find the shortest – or otherwise optimal – path between two points. Otherwise optimal paths could be paths that minimize the amount of turning, the amount of braking or whatever a specific application requires. Algorithms to find a shortest path are important not only in robotics, but also in network routing, video games and gene sequencing. Path-planning requires a map of the environment and the robot to be aware of its location with respect to the map. We will assume for now that the robot is able to localize itself, is equipped with a map, and capable of avoiding temporary obstacles on its way. How to create a map, how to localize a robot, and how to deal with uncertain position information will be major foci of the reminder of this class. The goals of this lecture are:

- Introduce suitable map representations.
- Explain basic path-planning algorithms ranging from Dijkstra, to A*, D* and RRT.
- Introduce variations of the path-planning problem, such as coverage path planning.

5.1 Map representations

In order to plan a path, we somehow need to represent the environment in the computer. We differentiate between two complementary approaches: discrete and continuous approximations. In a discrete approximation, a map is sub-divided into chunks of equal (e.g., a grid or hexagonal map) or differing sizes (e.g., rooms in a building). The latter maps are also known as topological maps. Discrete maps lend themselves well to a graph representation. Here, every chunk of the map corresponds to a vertex (also known as "node"), which are connected by edges, if a robot can navigate from one vertex to the other. For example a road-map is a topological map, with intersections as vertices and

roads as edges. Computationally, a graph might be stored as an adjacency or incidence list/matrix. A continuous approximation requires the definition of inner (obstacles) and outer boundaries, typically in the form of a polygon, whereas paths can be encoded as sequences of real numbers. Discrete maps are the dominant representation in robotics.

Currently the most common map is the occupancy grid map. In a grid map, the environment is discretized into squares of arbitrary resolution, e.g. 1cm x 1cm, on which obstacles are marked. In a probabilistic occupancy grid, grid cells can also be marked with the probability that they contain an obstacle. This is particularly important when the position of the robot that senses an obstacle is uncertain. Disadvantages of grid maps are their large memory requirements as well as computational time to traverse data structures with large numbers of vertices. A solution to the latter problem are topological maps that encode entire rooms as vertices and use edges to indicate navigable connections between them. There is no silver bullet, and each application might require a different solution that could be a combination of different map types.
There exist also every possible combination of discrete and continuous representation. For example, road maps for GPS systems are stored as topological maps that store the GPS coordinates of every vertex.

5.2 Path-Planning Algorithms

The problem to find a "shortest" path from one vertex to another through a connected graph is of interest in multiple domains, most prominently in the internet, where it is used to find an optimal route for a data packet. The term "shortest" refers here to the minimum cumulative edge cost, which could be physical distance (in a robotic application), delay (in a networking application) or any other metric that is important for a specific application.

One of the earliest and simplest algorithms is Dijkstra's algorithm. Starting from the initial vertex where the path should start, the algorithm marks all direct neighbors of the initial vertex with the cost to get there. It then proceeds from the vertex with the lowest cost to all of its adjacent vertices and marks them with the cost to get to them via itself if this cost is lower. Once all neighbors of a vertex have been checked, the algorithm proceeds to the vertex with the next lowest cost. A more detailed description of this algorithm is provided here. Once the algorithm reaches the goal vertex, it terminates and the robot can follow the edges pointing towards the lowest edge cost.

Looking at the animation to the right shows that the algorithm performs a lot of computations that are "obviously" not going in the right direction. They are obvious because

we have a birds-eye view of the environment, seeing the location of the obstacle, and we know the approximate direction of the goal. The latter extra knowledge that we have can be encoded using a heuristic function, a fancier word for a "rule of thumb". For example, we could give priority to nodes that have a lower estimated distance to the goal than others. For this, we would mark every node not only with the actual distance that it took us to get there (as in Dijkstra's algorithm), but also with the estimated cost "as the crows flies", for example by calculating the Euclidean distance or the Manhattan distance between the vertex we are looking at and the goal vertex. This algorithm is known as A*. Depending on the environment, A* might accomplish search much faster than Dijkstra's algorithm. The algorithm is provided in pseudo-code here. An extension of A* that addresses the problem of expensive re-planning when obstacles appear in the path of the robot, is known as D*. Unlike A*, D* starts from the goal vertex and has the ability to change the costs of parts of the path that include an obstacle. This allows D* to re-plan around an obstacle while maintaining most of the already calculated path.

A* and D* become computationally expensive when either the search space is large, e.g., due to a fine-grain resolution required for the task, or the dimensions of the search problem are high, e.g. when planning for an arm with multiple degrees of freedom. A more recent development known as Rapidly-Exploring Random Trees (RRT) addresses this problem by using a randomized approach that aims at quickly exploring a large area of the search space with iterative refinement. For this, the algorithm selects a random point in the environment and connects it to the initial vertex. Subsequent random points are then connected to the closest vertex in the emerging graph. The graph is then connected to the goal node, whenever a point in the tree comes close enough given some threshold. Although generally a coverage algorithm (see also below), RRT can be used for path-planning by maintaining the cost-to-start on each added point, and biasing the selection of points to occasionally falling close to the goal. RRT can also be used to take into account non-holonomic contraints of a specific platform when generating the next random way-point.

One way to implement this is to calculate the next random point by applying random values to the robot's actuators and use the forward kinematics to calculate the next point. Most recently, variations of RRT have been proposed that will eventually find an optimal solution. Nevertheless, although RRT quickly finds some solution, smooth paths usually require additional search algorithms that start from an initial estimate provided by RRT.

5.3 Robot embodiment

In order to deal with the physical embodiment of the robot, which complicates the path-planning process, the robot is reduced to a point-mass and all the obstacles in the environment are grown by half of the longest extension of the robot from its center. This representation is known as configuration space as it reduces the representation of the robot to its x and y coordinates in the plane. Most planning algorithms also do not consider the orientation of the robot. If this is desired, an additional dimension needs to be introduced into the configuration space.

5.4 Implementation

Path planning has been implemented using A* algorithm.

5.4.1 A* Algorithm

A* (pronounced as "A star") is a computer algorithm that is widely used in pathfinding and graph traversal. The algorithm efficiently plots a walkable path between multiple nodes, or points, on the graph. On a map with many obstacles, pathfinding from points to can be difficult. A robot, for instance, without getting much other direction, will continue until it encounters an obstacle, as in the path-finding example to the left below. However, the A* algorithm introduces a heuristic into a regular graph-searching algorithm, essentially planning ahead at each step so a more optimal decision is made. With A*, a robot would instead find a path in a way similar to the diagram on the right below. A* is an extension of Dijkstra's algorithm with some characteristics of breadth-first search (BFS).

Like Dijkstra, A* works by making a lowest-cost path tree from the start node to the target node. What makes A* different and better for many searches is that for each node, A* uses a function f(n) that gives an estimate of the total cost of a path using that node. Therefore, A* is a heuristic function, which differs from an algorithm in that a heuristic is more of an estimate and is not necessarily provably correct. A* expands paths that are already less expensive by using this function: f(n) = g(n) + h(n)

where

- f(n) = total estimated cost of path through node.
- g(n) = cost so far to reach node.
- h(n) = estimated cost from to goal. This is the heuristic part of the cost function, so it is like a guess.

In the grid above, A* algorithm begins at the start (red node), and considers all adjacent cells. Once the list of adjacent cells has been populated, it filters out those which are inaccessible (walls, obstacles, out of bounds). It then picks the cell with the lowest cost, which is the estimated f(n). This process is recursively repeated until the shortest path has been found to the target (blue node). The computation of is done via a heuristic that usually gives good results.

The calculation of can be done in various ways:

- The Manhattan distance (explained below) from node to the goal is often used. This is a standard heuristic for a grid.
- If heuristic = 0, A* becomes Dijkstra's algorithm, which is guaranteed to find a shortest path. The heuristic function must be admissible, which means it can never overestimate the cost to reach the goal. Both the Manhattan distance and heuristic = 0 are admissible.

Fig.5.1 represents pseudo code for A* algorithm.

```
make an openlist containing only the starting node
   make an empty closed list
   while (the destination node has not been reached):
       consider the node with the lowest f score in the open list
       if (this node is our destination node) :
           we are finished
       if not:
           put the current node in the closed list and look at all of its neighbo
rs
           for (each neighbor of the current node):
               if (neighbor has lower g value than current and is in the closed l
ist) :
                   replace the neighbor with the new, lower, g value
current node is now the neighbor's parent
               else if (current g value is lower and this neighbor is in the open
 list ) :
                   replace the neighbor with the new, lower, g value
                   change the neighbor's parent to our current node

               else if this neighbor is not in both lists:
                   add it to the open list and set its g
```

Figure 5.1: A* Algorithm

5.4.2 Obstacle detection and result

Given:
A set of test images containing

1. 10x10 grid, making 100 squares
2. Obstacles marked as black square
3. Objects defined by three features, viz. Shape, Size and Color

The squares are identified by the coordinate (x,y) where x is the column and y is the row to which the square belongs. Each square can be empty or have an Obstacle or have an Object.

Procedure:

- The given grid is processed using Open CV.
- Traversal of the grid takes place from square to square with help of cropped images of the grid squares.
- Lists are created for the co-ordinates with obstacle images and occupied grids.

Search algorithm to find the minimum path between 2 points takes three inputs.

1. The grid map
2. Start Point
3. End Point

This search algorithm is the A* algorithm.

Output:

1. The coordinates of occupied grid:

   The code returns a python list having ‘n’ python tuples, where ‘n’ denotes number of occupied grid in test image. Grid is to be considered occupied if either grid has an Obstacle or an Object. Each tuple has two elements, first element is the x-coordinate of an Obstacle/Object and second element is the y-coordinate of the Obstacle.

2. The minimum path:

   For each object in the test images, a matching object which is nearest to it is found using compare.ssim function from scikit-image. Object is said to be nearest to another Object, if length of path traversed between two objects is smallest. Traversal is done by moving either horizontally or vertically. The length of the path is determined by the number of moves made during traversal. A* search is used to find this shortest path.

The code return a python dictionary. Format for creating dictionary is as follows:

- Key for dictionary is a tuple - (x,y) coordinate of an Object.
- First element of dictionary is a tuple - (x,y) coordinate of an object nearest to it.
- Second element is a list of tuples having (x,y) coordinate of all grids traversed i.e all route path.
- Third element of dictionary should be number of moves taken for traversal.

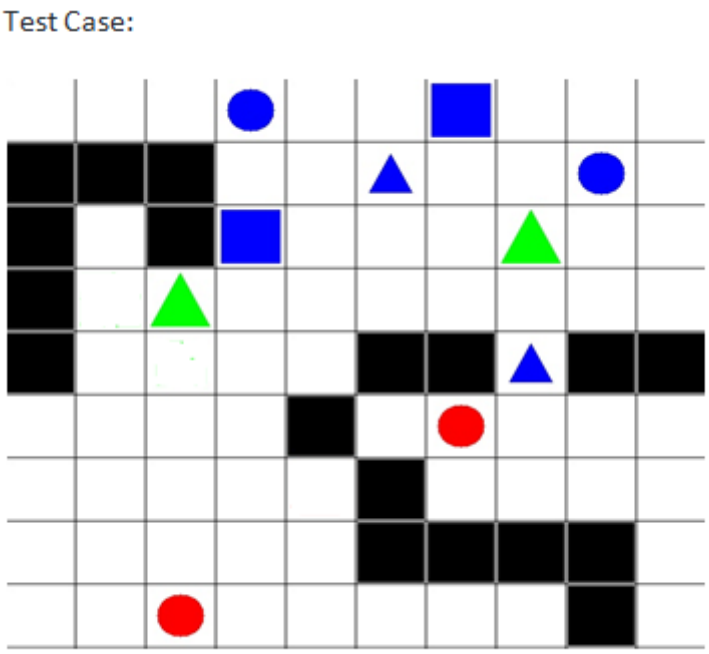


Figure 5.2: Testcase

Output:



Figure 5.3: Output

Chapter 6

Results and Observations

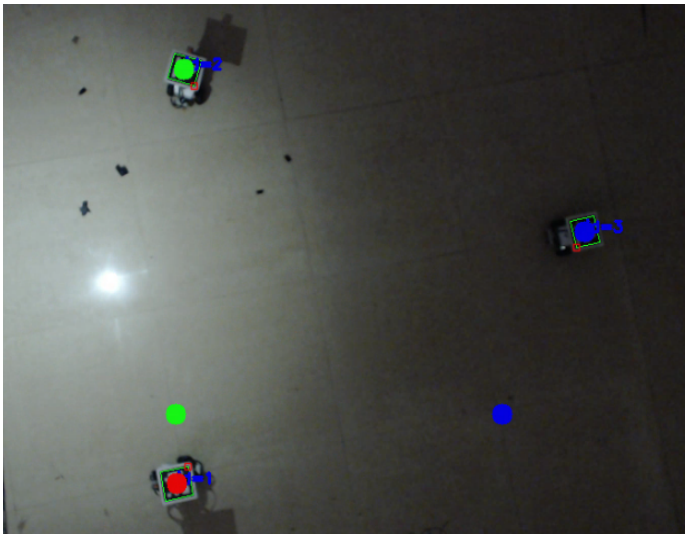


Figure 6.1: Initial Setup

The initial setup is shown in Fig.6.1. From the figure we can observe that there is one brightest spot and three robots in arbitrary locations.

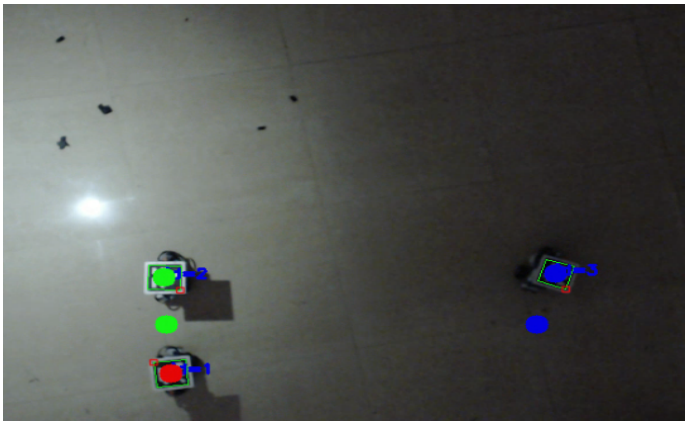


Figure 6.2: Iteration 1

Fig.6.2 shows the bots navigating towards the destination coordinates.

The light sensor measures the light intensity only after reaching the destination co-ordinates. It will not measure the intensity while the robot is in motion.

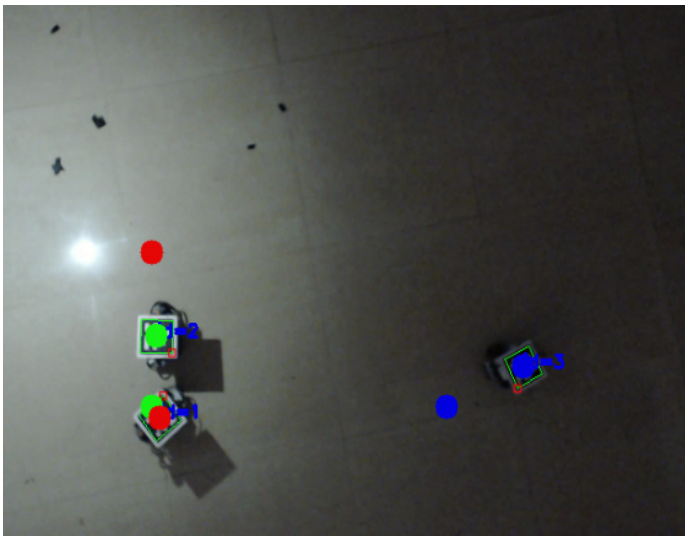


Figure 6.3: Obstacle detection

In Fig.6.3 we can observe that the green robot acts as an obstacle to the red robot. So, the ultrasonic sensor of the red robot detects an obstacle and decides to change its path to move to an intermediate position and then orients itself again towards the goal. The robot reaches subsequent intermediate positions till it does not encounter any obstacles. The algorithm is programmed in such a way that it detects both static and dynamic obstacles.

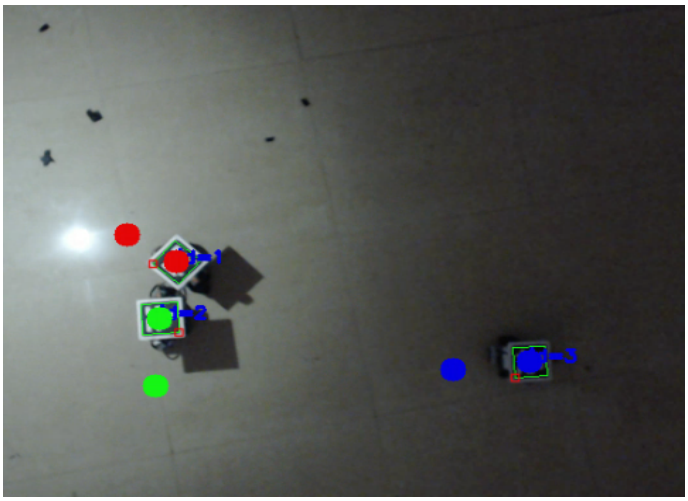


Figure 6.4: Iteration 2

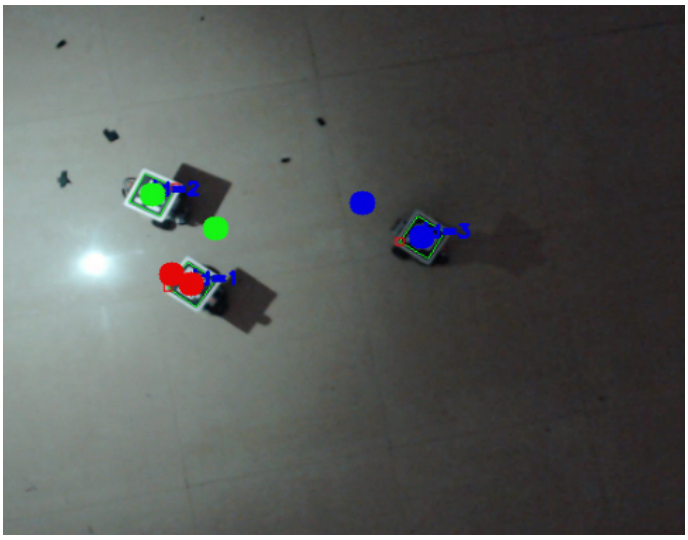


Figure 6.5: Iteration 3

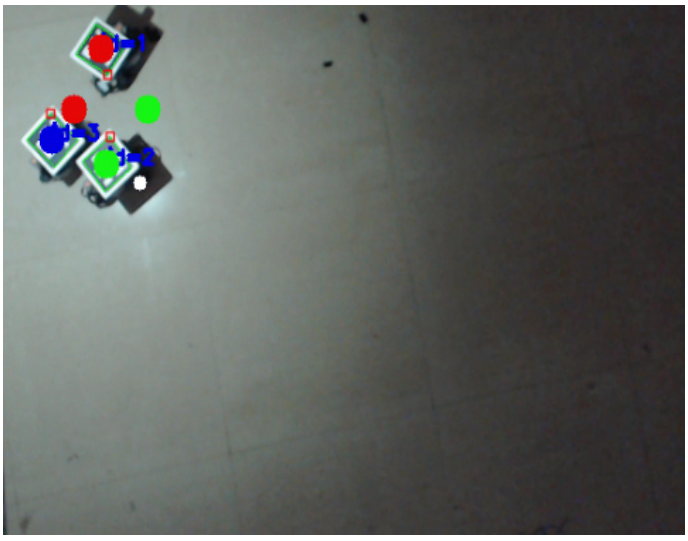


Figure 6.6: Final Result

After few iterations the robots successfully reaches the brightest spot which can be observed from Fig.6.6.

The table provides information about the time at which each robot reaches a particular destination in every iteration. This value is just based on one of the successful trials and hence cannot be generalized. The time taken by robots to reach the destination and the number of iterations to reach the target are not fixed. From the Fig.6.7 we can note that the robots reaches the brightest spot in 11 iterations. The fastest particle to reach the final target is the green robot which took 130s. The second fastest particle to reach the brightest spot is the blue robot which took 188s. The slowest the particle is red robot which took 264s.

Number of Iterations	Time taken by Robots to reach the brightest spot (In Sec)		
	Red Robot	Green Robot	Blue Robot
Iteration 1	0	22	26
Iteration 2	34	0	9
Iteration 3	1	0	5
Iteration 4	0	0	15
Iteration 5	0	40	11
Iteration 6	73	0	10
Iteration 7	64	0	7
Iteration 8	18	52	33
Iteration 9	0	12	6
Iteration 10	0	4	7
Iteration 11	74	0	59
Total Time taken	264	130	188

Figure 6.7: Time taken by three robots to reach the brightest spot

Chapter 7

Conclusion

We have implemented the PSO algorithm using three robots to find the brightest spot in a given space. We used an overhead camera to detect the ArUco markers to obtain the position of the robots at any given time. Ultrasonic sensor and light sensor were interfaced with the robots to detect the presence of obstacles and measure the light intensity respectively. The overhead camera and the three robots are connected to a central system via USB and Bluetooth respectively.The central system implements the PSO algorithm using the data given by the sensors and the overhead camera. Then the destination coordinates are given to the three robots.

From the results we can observe that the robots reach the brightest spot after a few iterations which took lesser time compared to randomized detection of the target.

Bibliography

[1] James M. Hereford,Michael Siebold, and Shannon Nichols, "Using the Particle Swarm Optimization Algorithm for Robotic Search Applications," Proceedings of the 2007 IEEE Swarm Intelligence Symposium (SIS 2007).

[2] Li-Chun Lai,Yen-Ching Chang,Jin-Tsong Jeng,Guan-Ming Huang,Wan-Ning Li, and Yi-Shan Zhang,"A PSO method for optimal design of PID controller in motion planning of a mobile robot,"Proceedings of 2013 International Conference on Fuzzy Theory and Its Application National Taiwan University of Science and Technology, Taipei, Taiwan, Dec. 6-8, 2013.

[3] Ying Pei,Wenbo Wang and Song Zhang "Basic Ant Colony Optimization," 2012 International Conference on Computer Science and Electronics Engineering.

[4] Ying-Tung Hsiao,Cheng-Long Chnang and Cheng-Chih Chien "Ant Colony Optimization for Best Path Planning," International Symposium on Communications and Information Technologies 2004 (ISCIT 2004)Sapporo, Japan, October 26-29,2004.

[5] Nitin Kumar Jhankal and Dipak Adhyaru, "Bacterial foraging optimization algorithm: A Derivative free technique,"INSTITUTE OF TECHNOLOGY, NIRMA UNIVERSITY, AHMEDABAD – 382 481, 08-10 DECEMBER, 2011 .

[6] Yichuan Shao and Hanning Chen, "Cooperative Bacterial Foraging Optimization," 2009 International Conference on Future BioMedical Information Engineering.

[7] Rishabh Gulati and Prashant Vats, "A Literature Review of Bee Colony Optimization Algorithms.," International Conference on Innovative Applications of Computational Intelligence on Power, Energy and Controls with their Impact on Humanity (CIPECH14) 28 29 November 2014

[8] Zhang Dongli, Guan Xinping, Tang Yinggan and Tang Yong, "An Artificial Bee Colony Optimization Algorithm based on Multi-exchange Neighborhood," 2012 Fourth International Conference on Computational and Information Sciences.

[9] Shubhendu Kumar Sarangi and Rutuparna Panda, ”A New Modified Firefly Algorithm for Function Optimization,” International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT) - 2016.

[10] Tian Yafei, Gao Weiming and Yan Shi ” An Improved Inertia Weight Firefly Optimization Algorithm and Application,”2012 International Conference on Control Engineering and Communication Technology.

[11] James L.Crowley ”Navigation for an Intelligent Mobile Robot,”IEEE JOURNAL OF ROBOTICS AND AUTOMATION, VOL. RA-I, NO. 1, MARCH 1985.

[12] Maxim A. Batalin, Gaurav S. Sukhatme and Myron Hattig ”Mobile Robot Navigation using a Sensor Network,”Proceedings of the 2004 IEEE International Conference on Robotics and Automation New Orleans, LA April 2004 .

[13] Sakshi Sharma, Rohit Salgotra and Urvinder Singh, ”An Enhanced Grey Wolf Optimizer for Numerical Optimization,” 2017 International Conference on Innovations in Information, Embedded and Communication Systems (ICIIECS).

[14] Benoit Martin, Julien Marot and Salah Bourennane ”Improved Discrete Grey Wolf Optimizer,” 2018 26th European Signal Processing Conference (EUSIPCO).